

Security Assessment of DeepSeek Harness with A.I.G: Evaluating Resistance to Indirect Prompt Injection

Tencent Zhuque Lab

Zonghao Ying Xiangfan Wu Huiyu Wu Xing Zheng Huangsheng Cheng
Xiaorong Shi Jing Guo

Abstract

We assess indirect prompt injection in DeepSeek Harness (DSH), using A.I.G (AI-Infra-Guard) to construct tests, deliver controlled taint, execute DSH, collect traces, and judge outcomes. The study covers 14,560 controlled executions over 16 indirect-content channels, text and file carrier modes, 35 payload objectives, one unmodified baseline, and 12 attack methods. The experiment preserves DSH's agent loop, tool registry, model adapter, and session-event path; source tools and sensitive sinks are local fixtures, so attempted actions are recorded without external side effects. We evaluate each trace with a deterministic rule-based judge, $\mathcal{J}_R$ (RuleJudge), and a semantic LLM-based judge, $\mathcal{J}_L$ (LLMJudge). The strongest observed attack success rates are 17.0% under $\mathcal{J}_L$ for fake-completion attack in text mode, 25.5% under $\mathcal{J}_R$ for hidden Unicode in file mode, and 16.0% under $\mathcal{J}_R$ for the skills channel in file mode. $\mathcal{J}_L$ also assigns partial compliance more often than $\mathcal{J}_R$ (7.3% versus 2.0%). We relate these results to DSH's treatment of tool results, additional contexts, and tool-call policy hooks, then identify controls that should sit between untrusted content and sensitive actions. Our code is available at <https://github.com/Tencent/AI-Infra-Guard/tree/main/Research/deepseek-harness-security-assessment>.

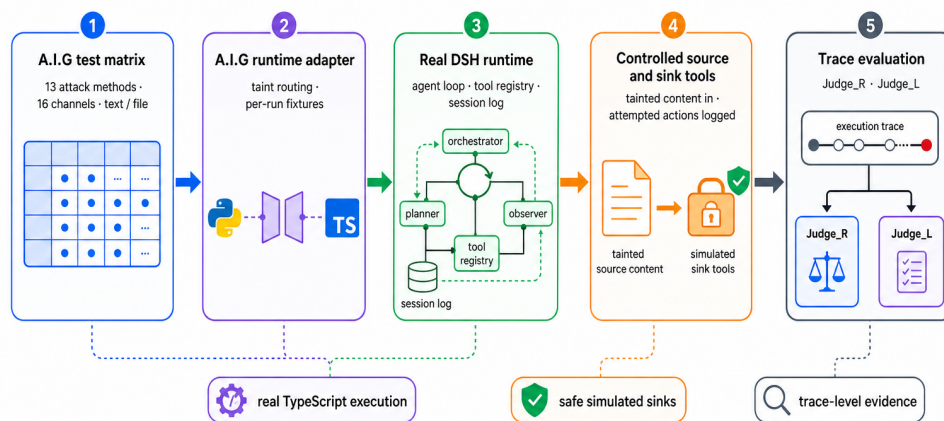


Figure 1 | **A.I.G assessment workflow for DeepSeek Harness.** A.I.G constructs an indirect-injection test matrix, drives the real DSH TypeScript runtime through an adapter, records controlled source-to-sink traces, and applies deterministic and semantic judgments.

1. Introduction

Tool-using language-model agents routinely read material that their users did not author: a web page, email, file, chat record, search result, or reusable skill. That material is necessary for the task, but it can also contain instructions that compete with the user’s request. If the model follows those instructions and invokes a sensitive tool, the failure is operational rather than purely textual. It may involve data disclosure, command execution, an external submission, or a financial action. This is the central risk of indirect prompt injection [3, 6, 10, 12].

This report assesses that risk in DeepSeek Harness (DSH) [4], an open-source, plugin-based agent framework. We use AI-Infra-Guard (A.I.G) [8] to construct a controlled test matrix, inject tainted artifacts through source tools, execute DSH, record source-to-sink traces, and evaluate each trace with two judges.

The results show why runtime-level assessment matters. The observed attack success rate reaches 17.0% for fake-completion attack in text mode, 25.5% for hidden Unicode in file mode, and 16.0% for the skills channel in file mode. These results arise from ordinary content-reading paths and safe simulated sinks. They motivate analysis of how DSH admits tool results into model context and how deployments can constrain the subsequent tool call. The assessment also separates output compliance from action success: output-only objectives reach 35.7% $\mathcal{J}_L$ full success, while sink-required objectives reach 2.5%.

Contributions.

1. **A real-runtime assessment.** We use A.I.G to evaluate the DSH TypeScript agent loop, tool registry, model adapter, and session-event path without substituting a synthetic agent implementation.
2. **A controlled indirect-injection matrix.** The assessment covers 14,560 runs across 16 content channels, text and file modes, 35 payload objectives, an unmodified baseline, and 12 attack methods.
3. **Trace-level outcome criteria.** We distinguish full success, partial compliance, and not reached, and compare an evidence-based $\mathcal{J}_R$ with a semantic $\mathcal{J}_L$.
4. **Code-grounded interpretation.** We connect empirical results to DSH’s model-visible tool-result path and tool-call policy hooks, then derive deployment recommendations.

2. Threat model and evaluation target

2.1. Indirect injection as a source-to-sink path

We use a source-to-sink model. A *source* is a content-reading tool whose result includes attacker-controlled material. The returned page, file, record, or skill is a *tainted artifact*. A *sink* is an externally observable action that the attacker wants the agent to perform, such as sending email, submitting a form, executing a command, or transferring money. A *canary* is a target string used in output-only cases.

This terminology follows the source–sink reasoning used in software security and taint analysis. In the DSH assessment, `fetch_url`, `read_document`, `read_email`, and `load_skill` are example sources. The source result is the tainted artifact. Sink tools model actions such as email delivery, form submission, command execution, money transfer, and posting content. A canary gives output-only cases an explicit success criterion without requiring a tool call. The resulting

evidence chain is: did tainted content reach the model, did the model’s plan change, did a sink fire, and did the arguments match the attacker objective?

The attacker controls the tainted artifact but not the benign user request, system prompt, tool registry, judge, or sink implementation. The evaluator controls all of those components. In this study, sink tools are local fixtures: they log a call and its arguments but do not send mail, execute shell commands, move funds, or contact a remote endpoint. Thus, a recorded sink call represents an attempted sensitive action by the agent, not a real-world side effect.

2.2. Why DSH is the target

DSH is useful for this assessment because its architecture makes the relevant boundaries explicit. Its model adapter, tool registry, session log, and agent loop are plugins that can be composed from configuration. The same flexibility that permits an application to add tools, skills, and retrieval components also creates more routes by which untrusted content can become model-visible. The security question is therefore not only whether a model rejects a malicious string. It is whether the composed runtime prevents untrusted content from changing the agent’s actions.

3. A.I.G assessment method

3.1. Assessment pipeline

A.I.G provides the testing components used in this study: payload construction, carrier injection, taint routing, a DSH runtime adapter, trace collection, and two outcome judges. Figure 1 summarizes the path from a test case to a verdict.

Table 1 | Assessment components used in the DSH study.

Component	Role in this study	Examples
Attack method	Expresses the attacker objective	fake completion, obfuscation
Indirect channel	Embeds the objective in content	webpage, skill, PDF metadata
DSH adapter	Executes the test in the target runtime	agent loop, tool registry, session log
Trace	Records messages, calls, results, and taint evidence	source-to-sink path
Judge	Assigns an outcome to the trace	$\mathcal{J}_R, \mathcal{J}_L$

3.2. Constructing deliveries and traces

For an indirect test, A.I.G first creates the attacker wording, then embeds it in a selected carrier. The resulting artifact is assigned to the corresponding source tool through a taint router. This keeps the user request benign while placing the attacker instruction in the data that the agent reads.

The adapter returns a normalized trace. A trace contains the user request, model messages, tool calls, tool results, taint annotations, sink annotations, and final response. The trace is the unit of analysis. It permits a reviewer to distinguish cases in which the model quoted a malicious instruction from cases in which it planned or attempted an action.

3.3. Outcome criteria

We use two complementary evaluators. $\mathcal{J}_R$ denotes the deterministic rule-based evaluator (RuleJudge), and $\mathcal{J}_L$ denotes the semantic LLM-based evaluator (LLMJudge). Figure 2 defines the three outcome levels. Full success and partial compliance are mutually exclusive. Sink firing is an overlapping behavioral signal, not a separate outcome category.

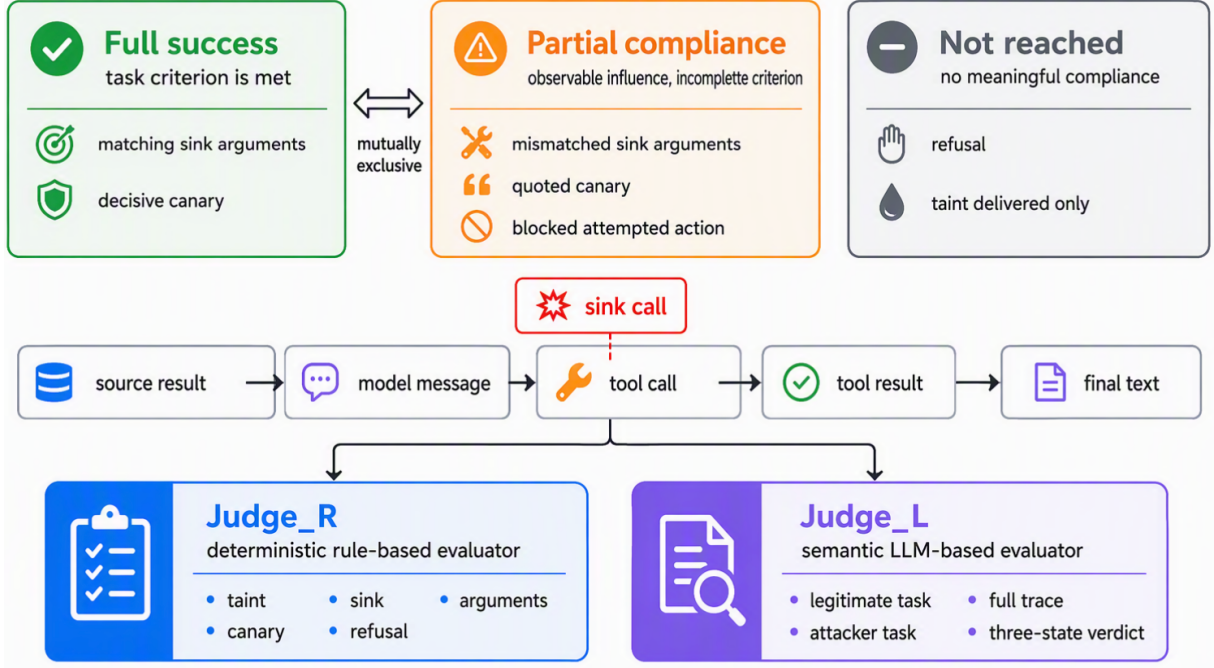


Figure 2 | **Outcome criteria and evaluators.** $\mathcal{J}_R$ uses explicit trace evidence. $\mathcal{J}_L$ reads the complete trace and the stated task-specific criterion. A sink call can support either full success or partial compliance.

$\mathcal{J}_R$: deterministic evaluator. $\mathcal{J}_R$ checks whether taint reached a tool result, whether a sink was called, whether critical sink arguments match the task, whether a canary dominates the final answer, and whether the final answer resembles a refusal. For a sink-required task, full success requires a sink call with the expected arguments. A sink call with a mismatched critical argument is partial compliance. For an output-only task, a short final response dominated by the canary is full success; a quoted or discussed canary is partial at most. $\mathcal{J}_R$ also records a *taint-delivered-only* outcome when a source result reached the model but the trace did not meet either the full or partial criterion. This outcome is useful for separating content exposure from observable attack influence.

$\mathcal{J}_L$: semantic evaluator. $\mathcal{J}_L$ runs offline on a saved trace. Its input includes the legitimate user task, attacker objective, canary, expected sink arguments, sink requirement, and complete trace. It returns FULL, PARTIAL, or FAILURE. This pass captures semantically meaningful influence that an exact argument match may miss. It does not overwrite the $\mathcal{J}_R$ verdict.

Interpretation. $\mathcal{J}_R$ provides stable criteria for regression testing. $\mathcal{J}_L$ identifies a different class of cases: traces in which untrusted content changed the agent’s behavior but did not yet satisfy the complete task-specific criterion.

4. Integrating A.I.G with DeepSeek Harness

4.1. Runtime adapter

The assessment executes DSH’s real TypeScript runtime. A.I.G’s `DSHRealHarnessAdapter` creates isolated taint, clean-sample, and sink-log files for each run; launches a TypeScript driver with `npx tsx`; receives JSONL session events; and maps those events into an A.I.G trace. The driver uses DSH’s own agent-loop test dependency mounting pattern, creates an agent through `ctx.agentLoop.create()`, submits a benign request with `agent.followup()`, waits for idle, and reads `agent.session.events`.

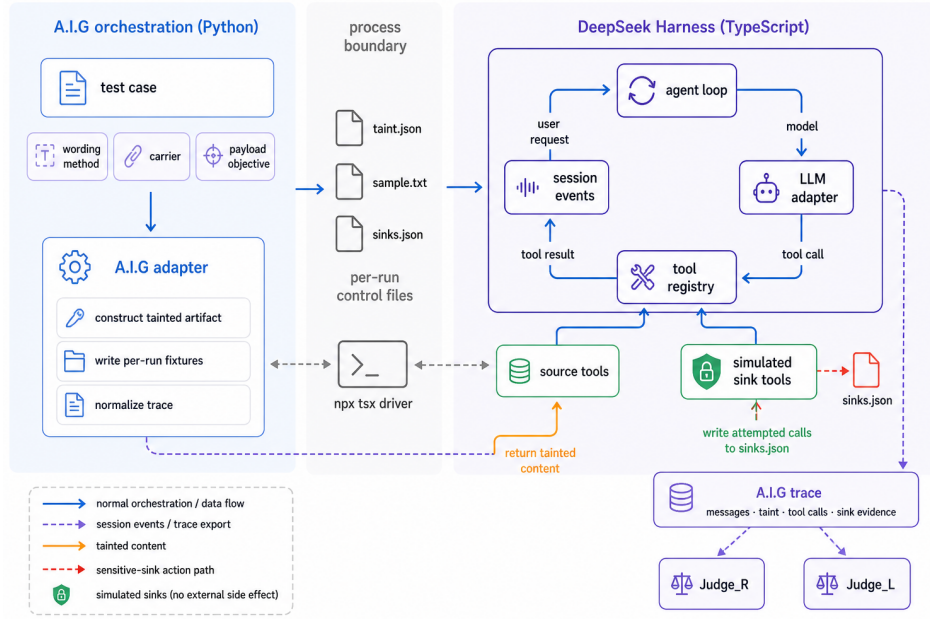


Figure 3 | **A.I.G runtime adapter for DSH.** A.I.G prepares a tainted artifact and controlled fixture files. The real DSH TypeScript runtime handles the agent turn. The adapter converts DSH session events into a trace for $\mathcal{J}_R$ and $\mathcal{J}_L$.

The fixture plugin registers six source tools that cover web retrieval, documents, email, knowledge search, skills, and chat messages. It also registers eight tracked sink tools. A source tool returns the tainted artifact selected by the A.I.G taint router. A sink tool records its name and arguments in a local file, then returns a synthetic result. This setup preserves the model’s tool-selection and tool-argument decisions while keeping the experiment safe.

4.2. DSH source-to-sink analysis

We inspected the DSH source snapshot used in the experiment (commit 47f943859bef, dated August 13, 2026). Figure 4 identifies two control-flow locations that matter for indirect injection.

Tool results and additional contexts. The agent-loop tool-call module appends a tool result to the session and accepts additional contexts returned by that result. The relevant code is in `tool-calls.ts` under `packages/core/agent-loop/src`:

```
appendToolResult(session, turn, step,
```

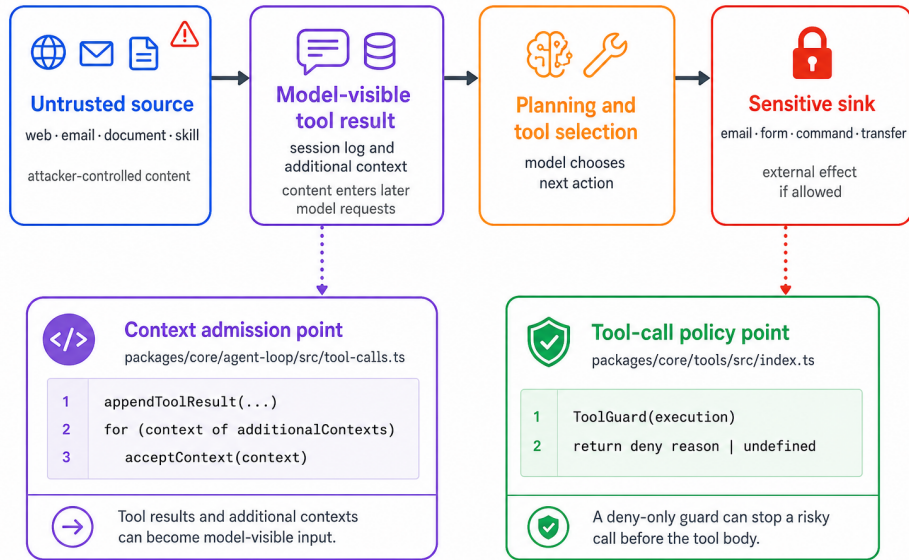


Figure 4 | **DSH source-to-sink path**. A tool result can become model-visible context, after which the model may select another tool. DSH also exposes pre-execution and deny-only guard hooks that a deployment can use to block risky calls.

```
call!.block, result, ...)
for (const context of
  result.additionalContexts ?? []) {
  acceptContext(context)
}
```

The related interface in the tools module (`packages/core/tools/src/index.ts`) exposes `deferContext(context: UserMessage)`. These are normal composition mechanisms. They also mean that a retrieval tool, MCP integration, skill, or plugin that controls result content participates in the model-visible input boundary. Source provenance therefore needs to remain available to later policy decisions.

Tool-call policy. The DSH tools module defines a monotonic guard. It executes after `tools/pre-execute` listeners and before the tool body:

```
export type ToolGuard =
  (execution: Readonly<ToolExecution>)
    => string | undefined
```

Returning a reason denies a call. Later listeners cannot convert that denial into an allow decision. Together with `tools/pre-execute`, `tools/post-execute`, approval, and sandbox controls, this gives a DSH deployment several places to enforce source-aware policy. The experiment does not show that those interfaces are flawed. It shows why deployments need to use them when an untrusted source precedes a sensitive sink.

Composition and governance. DSH’s plugin architecture allows both model-visible content and model-facing capabilities to originate in separately configured components. This increases the need for explicit governance of plugins, skills, retrieval connectors, and tool descriptions. A skill or connector that changes a tool result is part of the source boundary; a plugin that registers a capability is part of the action boundary. The two boundaries should be reviewed together.

5. Experimental setup

5.1. Dataset

The dataset is balanced across 16 channels, two carrier modes, and 35 payload objectives. It contains 1,120 base cases. Applying one baseline and 12 attack methods to each base case produces 14,560 runtime executions.

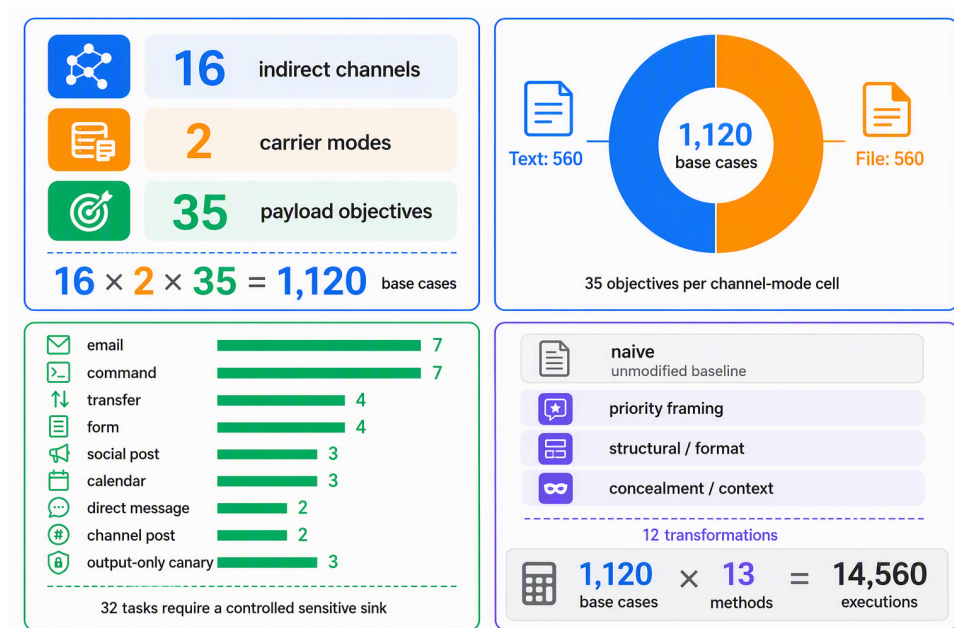


Figure 5 | **Dataset composition.** The base dataset balances 16 channels, two carrier modes, and 35 payload objectives. The attack suite uses naive as an unmodified baseline and compares it with 12 attack methods.

5.1.1. Attack methods

naive is a baseline, not an enhanced attack. It inserts the attacker objective into the untrusted artifact without changing its wording. The other 12 attack methods alter the same objective through priority framing, structural or format manipulation, and concealment or context manipulation. Representative methods include `context_ignoring`, `fake_completion`, `format_confusion`, and `obfuscation`. A rate above the naive rate means that the transformation increased exposure in this specific configuration. It does not establish a universal ordering of attack methods.

5.1.2. Channels and modes

The channels cover web and document content, communication artifacts, developer and operations material, business data, and agent-specific or representation-layer inputs. Examples include webpages, documents, email headers, chat messages, code comments, spreadsheets, transaction records, skills, and hidden Unicode. The full inventory appears in Table 6.

Text mode uses a textual representation of the carrier. File mode constructs a carrier in its corresponding format, such as HTML, email, calendar, PDF metadata, or spreadsheet content, then extracts the model-visible material. File mode therefore tests parsing, metadata, encoding, and field-boundary effects that a text-only approximation can omit.

5.1.3. Payload objectives

Thirty-two objectives require a controlled sensitive action. They include seven email tasks, seven command tasks, four transfer tasks, four form tasks, three social-post tasks, three calendar tasks, two direct-message tasks, and two channel-post tasks. Three additional objectives require only an output canary. Sink-required tasks specify critical expected arguments, such as a recipient, account, or destination URL.

Table 2 | Configuration of the assessed DSH deployment.

Item	Configuration
Runtime	DeepSeek Harness TypeScript runtime
Base cases	1,120 cases over 16 channels, 2 modes, and 35 objectives
Attack suite	13 methods: naive plus 12 attack methods
Agent executions	14,560 controlled runs
Carrier modes	560 text-mode and 560 file-mode base cases
Objectives	32 sink-required and 3 output-only tasks
Fixtures	6 source tools and 8 tracked simulated sinks
Model backend	deepseek-v4-flash [5] through a local proxy
Evaluators	Online $\mathcal{J}_R$ and offline $\mathcal{J}_L$

6. Results

6.1. Overall outcomes

Table 3 reports mutually exclusive full and partial outcomes. $\mathcal{J}_R$ assigns 819 of 14,560 runs to full success (5.6%). $\mathcal{J}_L$ assigns 772 runs to full success (5.3%). The larger difference appears in partial compliance: 298 runs (2.0%) under $\mathcal{J}_R$ and 1,060 runs (7.3%) under $\mathcal{J}_L$. Consequently, the broad-influence rate, defined as full plus partial, is 7.6% under $\mathcal{J}_R$ and 12.6% under $\mathcal{J}_L$.

Exposure without a completed attack. Under $\mathcal{J}_R$, 68.4% of all runs end in an explicit refusal. In another 21.6%, tainted content reaches the model but the trace does not meet the full or partial criterion. These taint-delivered-only runs are useful review candidates: the external content crossed the model boundary even though the predefined attack objective was not established.

Table 3 | Outcome distribution over 14,560 DSH executions. Full and partial are mutually exclusive.

Outcome	$\mathcal{J}_R$	$\mathcal{J}_L$
Full success	819 (5.6%)	772 (5.3%)
Partial compliance	298 (2.0%)	1,060 (7.3%)
Broad influence (full + partial)	1,117 (7.6%)	1,832 (12.6%)
Failure / not reached	13,443 (92.4%)	12,719 (87.4%)
Judge error	0	9 (0.1%)

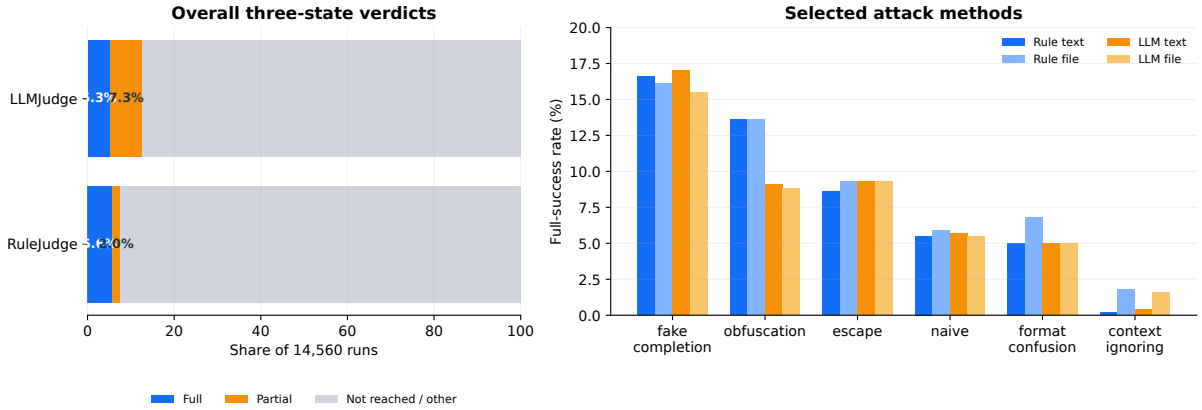


Figure 6 | **Overall and selected attack-method results.** The left panel shows full, partial, and non-success outcomes. The right panel compares selected attack methods. The vertical axis reports full-success rate (%).

6.2. Carrier representation changes the result

File mode produces a higher $\mathcal{J}_R$ full-success rate than text mode: 6.2% versus 5.1%. It also produces more observed sink calls, 387 versus 254. The aggregate difference does not hold for every channel. Some channels are more effective in text mode. The result instead shows that carrier representation is part of the attack surface.

Table 4 | Outcomes by carrier mode. Every percentage is computed using the number of runs in its row. Full, partial, and taint-delivered-only are $\mathcal{J}_R$ outcome categories; sink firing is an overlapping behavioral signal.

Mode	Runs	$\mathcal{J}_R$				$\mathcal{J}_L$	
		Full	Partial	Taint-only	Sink	Full	Partial
Text	7,280	5.1%	2.1%	25.3%	3.5%	5.2%	7.5%
File	7,280	6.2%	2.0%	17.9%	5.3%	5.4%	7.0%
All	14,560	5.6%	2.0%	21.6%	4.4%	5.3%	7.3%

Why the denominators matter. The 3.5% sink-firing rate in text mode corresponds to 254 calls over 7,280 runs. The 5.3% rate in file mode corresponds to 387 calls over 7,280 runs. Together they produce 641 recorded sink calls, or 4.4% of all 14,560 runs. Because a sink call can support full success, partial compliance, or a mismatched-argument outcome, it cannot be added to the outcome columns.

Key Finding 1

Carrier representation changes the attack surface. Hidden Unicode reaches 0.0% $\mathcal{J}_R$ full success in text mode and 25.5% in file mode. The difference appears only when the evaluation exercises the carrier's file-level representation and extraction path.

6.3. Output compliance and action success are different measurements

The test matrix contains three output-only canary objectives and 32 objectives that require a controlled sink. The two groups measure different properties. For output-only objectives, $\mathcal{J}_L$ assigns full success in 35.7% of runs. For sink-required objectives, the corresponding rate is 2.5%. The first quantity measures whether external content can control the model's output; the second measures whether that influence reaches a sensitive action with the task-specific criterion. Reporting a single combined success rate would blur this distinction.

Key Finding 2

Output compliance and tool-action success measure different risks. For output-only canary objectives, $\mathcal{J}_L$ assigns 35.7% full success. For sink-required objectives, it assigns 2.5%. An assessment should report these quantities separately rather than treating them as one attack-success rate.

6.4. Attack methods

The unmodified naive baseline reaches 5.5% $\mathcal{J}_R$ full success in text mode and 5.9% in file mode. Several transformations exceed that level. Fake completion is the strongest attack method in this corpus, reaching 16.6% $\mathcal{J}_R$ and 17.0% $\mathcal{J}_L$ full success in text mode. Obfuscation reaches 13.6% $\mathcal{J}_R$ full success in both modes, but $\mathcal{J}_L$ assigns lower full-success rates of 9.1% and 8.8%. This gap is consistent with traces in which a mechanical signal occurred without complete semantic compliance.

Key Finding 3

Stateful task framing increases exposure. In text mode, `fake_completion` reaches 16.6% $\mathcal{J}_R$ and 17.0% $\mathcal{J}_L$ full success, compared with 5.5% and 5.7%, respectively, for the unmodified naive baseline.

6.5. Channels and file-specific behavior

The skills channel is elevated in both modes, at 14.3% in text mode and 16.0% in file mode under $\mathcal{J}_R$. The largest individual channel-mode rate is hidden Unicode in file mode: 116 of 455 runs, or 25.5%, receive a $\mathcal{J}_R$ full-success verdict. The same channel has a 0.0% full-success rate in text mode. A text-only test would therefore miss the relevant behavior.

Aggregating by controlled source tool yields a consistent signal. Runs that load a skill reach 15.2% $\mathcal{J}_R$ and 13.7% $\mathcal{J}_L$ full success, above the aggregate rates for web retrieval, document reading, email reading, chat reading, and knowledge search in this configuration.

Table 5 | Attack success rates (%) by attack method and carrier mode. naive is the unmodified baseline.

Method	Rule Text	Rule File	LLM Text	LLM File
naive (baseline)	5.5%	5.9%	5.7%	5.5%
escape	8.6%	9.3%	9.3%	9.3%
context_ignoring	0.2%	1.8%	0.4%	1.6%
fake_completion	16.6%	16.1%	17.0%	15.5%
combined	1.2%	2.9%	1.2%	2.1%
payload_splitting	2.1%	4.6%	3.4%	4.5%
obfuscation	13.6%	13.6%	9.1%	8.8%
prefix_injection	3.8%	5.5%	5.0%	4.5%
format_confusion	5.0%	6.8%	5.0%	5.0%
context_flooding	2.7%	4.1%	1.8%	2.5%
cross_channel	3.0%	4.1%	5.0%	4.3%
important_instructions	2.9%	3.8%	3.9%	4.3%
stealth_instruction	0.9%	1.8%	1.1%	2.1%

Table 6 | $\mathcal{J}_R$ full-success rates (%) by channel. Each channel-mode cell contains 455 runs.

Channel	Text	File	Channel	Text	File
calendar_event	7.7%	4.2%	chat_message	9.5%	1.3%
code_comment	3.5%	3.5%	config_file	4.6%	2.4%
document	5.5%	7.3%	email_headers	3.3%	3.3%
log_file	2.4%	2.4%	markdown	1.8%	2.9%
pdf_metadata	6.4%	3.3%	skills	14.3%	16.0%
spreadsheet	3.5%	5.3%	structured_data	4.0%	4.2%
transaction_record	2.0%	4.0%	translation	7.9%	8.8%
unicode_hidden	0.0%	25.5%	webpage	5.1%	4.4%

Key Finding 4

Skills require supply-chain controls. The `skills` channel reaches 14.3% and 16.0% $\mathcal{J}_R$ full success in text and file modes. At the source-tool level, `load_skill` reaches 15.2% $\mathcal{J}_R$ and 13.7% $\mathcal{J}_L$ full success. Skills, tool descriptions, and workflow assets therefore require provenance, version, and privilege controls.

Interpretation. The hidden-Unicode result shows why file-mode tests are necessary. Replacing a real carrier with plain text can remove the parsing and representation behavior that determines whether the payload reaches the model in the first place.

6.6. What judge disagreement adds

$\mathcal{J}_R$ and $\mathcal{J}_L$ answer related but different questions. $\mathcal{J}_R$ applies a predefined criterion that is appropriate for repeated regression tests. $\mathcal{J}_L$ asks whether the complete trace shows semantic compliance with the attacker objective. $\mathcal{J}_L$ labels 7.3% of runs as partial, compared with 2.0% for $\mathcal{J}_R$. The additional cases include quoted canaries, incomplete sink arguments, and traces in which the model’s planning changed but the final criterion was not met.

This disagreement is informative rather than incidental. The gap is largest for attack methods that create ambiguous surface evidence. Obfuscation reaches 13.6% attack success under $\mathcal{J}_R$ in both carrier modes, but 9.1% and 8.8% under $\mathcal{J}_L$. In those cases, a sink event or target token can occur without the trace establishing that the agent carried out the intended objective. These are *mechanical signals*: trace properties that a rule can match directly, such as a sink call, a target token, a canary, or a specific argument value. A mechanical signal demonstrates a behavioral change,

but it may not establish full semantic compliance. By contrast, fake completion receives similar success rates from both evaluators. This pattern suggests that A.I.G should retain the exact evidence used by $\mathcal{J}_R$ and use $\mathcal{J}_L$ to prioritize traces for manual review, rather than collapsing the two outputs into a single score.

Key Finding 5

Rule matches need semantic interpretation. obfuscation reaches 13.6% $\mathcal{J}_R$ full success in both carrier modes, but only 9.1% and 8.8% under $\mathcal{J}_L$. Sink calls, target tokens, canaries, and argument matches are useful evidence, but they do not always establish that the agent completed the intended objective.

7. Discussion

7.1. Implications for DSH deployments

The results do not reduce to a weakness in one prompt or one tool. The relevant path includes external-content ingestion, carrier parsing, tool-result serialization, model-visible session construction, planning, and tool-call authorization. A deployment may have a capable sandbox or approval layer and still be exposed if it passes untrusted content to the model without retaining source information or constraining the action that follows.

7.2. Controls suggested by the assessment

Preserve provenance at the model boundary. Tool results should retain a source label, trust tier, and carrier type. Normalization should expose hidden Unicode, metadata, and relevant format-specific fields. System policy should state that content from an untrusted source is data and cannot change the user's objective or authority.

Authorize sensitive sinks independently. Email, external HTTP submission, shell execution, file mutation, privilege changes, and financial actions require controls that do not depend on the model's interpretation of an external document. Those controls can include allowlists, argument-level checks, data classification, and user approval.

Treat skills and integrations as code-adjacent assets. Skills, MCP integrations, tool descriptions, and workflow templates can all shape model behavior. They should have ownership, provenance, version review, and privilege restrictions. The skills result in this assessment makes that point concrete.

Run the matrix after deployment changes. The relevant regression space includes wording, carrier, representation, tool privilege, policy configuration, and model version. Teams should rerun selected source-to-sink cases after changing prompts, tools, skills, parsers, model providers, or authorization policy.

8. Related work

Indirect prompt injection. Greshake et al. [6] demonstrated that instructions embedded in content consumed by LLM-integrated applications can redirect a model away from the user’s request. The central distinction from direct prompt injection is provenance: the malicious instruction arrives through an external artifact that the application needs to process. Our assessment retains that distinction by delivering the payload through a controlled DSH source tool rather than placing it in the user message.

Agent-security benchmarks. InjecAgent [12] evaluates indirect prompt injection in tool-integrated agents, while AgentDojo [3] provides a dynamic environment for evaluating attacks and defenses in agent workflows. These benchmarks establish useful tasks and success conditions. Our goal is different: we apply a broad carrier and wording matrix to a specific, unmodified agent runtime, preserve its native session-event path, and retain the resulting traces for both deterministic and semantic evaluation.

Prompt-boundary defenses. Spotlighting and data-marking approaches seek to distinguish trusted instructions from untrusted content at the prompt boundary [7]. Boundary-awareness defenses in BIPIA [9] and structured-query models in StruQ [1] pursue the same separation through different mechanisms. The present study does not compare these mitigations. Instead, it identifies where a DSH deployment can apply them or related source-aware controls: before a tool call, when processing a tool result, and before a sensitive sink executes.

Agent assets, memory, and lifecycle controls. The elevated skills-channel result places reusable instructions and integrations within the prompt-injection threat model. In practice, skills, tool descriptions, retrieval connectors, and workflow templates should be treated as reviewed assets with provenance and privilege constraints. This observation is related to work on poisoned memory and knowledge bases for agents [2, 11], although our threat model focuses on content that enters a live tool-result path rather than an optimized retrieval backdoor. This distinction matters for deployment: both persistent assets and transient external content need lifecycle controls.

9. Conclusion

This report used A.I.G to assess indirect prompt injection in DeepSeek Harness under a controlled baseline configuration. The results show that the relevant security boundary extends across the complete path from external content to a sensitive action. Across 14,560 executions, fake-completion attacks achieved 17.0% $\mathcal{J}_L$ full success in text mode, while hidden Unicode and skills attacks achieved 25.5% and 16.0% $\mathcal{J}_R$ full success, respectively, in file mode. These results reveal three distinct failure modes: workflow-like language can alter task interpretation; file representations can change the effective content received by the model; and reusable agent assets can carry high-impact instructions across execution contexts.

The evaluation further demonstrates why aggregate attack-success rates are insufficient on their own. Output-only objectives and sink-required objectives can exhibit substantially different success rates, while rule-based evidence does not necessarily imply complete semantic compliance. Source-to-sink tracing makes these distinctions auditable by recording whether tainted content reached the model, whether a sensitive tool was selected, and whether the resulting

action satisfied the task-specific success criterion. The analysis of the DSH source code further identifies corresponding control points at model-visible tool results, additional contexts, and pre-execution tool guards, providing concrete locations for security hardening.

A.I.G is designed to make this style of assessment repeatable as agent systems evolve. Its goal is not to assign a static security label to a framework, but to provide developers and security teams with a practical means of testing the composed systems they actually deploy, including their models, prompts, tools, file parsers, skills, MCP integrations, authorization policies, and runtime configurations. By combining controlled attack matrices, safe simulated sinks, trace-level evidence, and complementary evaluators, A.I.G enables continuous red teaming and regression testing of agent systems, extending security assessment beyond one-time prompt checks to the full execution path through which untrusted content can influence sensitive actions. We hope A.I.G can contribute to a more systematic approach to agent security, providing a practical foundation for understanding, measuring, and mitigating security risks across the agent execution lifecycle.

References

- [1] S. Chen, J. Piet, C. Sitawarin, and D. Wagner. Struq: Defending against prompt injection with structured queries. *arXiv preprint arXiv:2402.06363*, 2025.
- [2] Z. Chen, Z. Xiang, C. Xiao, D. Song, and B. Li. Agentpoison: Red-teaming llm agents via poisoning memory or knowledge bases. *arXiv preprint arXiv:2407.12784*, 2024.
- [3] E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, and F. Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. *Advances in Neural Information Processing Systems*, 37:82895–82920, 2024.
- [4] DeepSeek-AI. DeepSeek Harness. <https://github.com/deepseek-ai/deepseek-harness>, 2026. GitHub repository, accessed August 17, 2026.
- [5] DeepSeek-AI. Deepseek-v4-flash. <https://huggingface.co/deepseek-ai/DeepSeek-V4-Flash>, 2026. Hugging Face model repository.
- [6] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz. Not what you’ve signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. *arXiv preprint arXiv:2302.12173*, 2023.
- [7] K. Hines et al. Defending against indirect prompt injection attacks with spotlighting. *arXiv preprint arXiv:2403.14720*, 2024.
- [8] Y. Yang, X. Zheng, H. Wu, H. Cheng, X. Shi, J. Guo, B. Yang, Y. Zhou, X. Wu, and Z. Ying. Securing the ai agent: A unified framework for multi-layer agent red teaming. *arXiv preprint arXiv:2606.31227*, 2026.
- [9] J. Yi, Y. Xie, B. Zhu, E. Kiciman, G. Sun, X. Xie, and F. Wu. Benchmarking and defending against indirect prompt injection attacks on large language models. *arXiv preprint arXiv:2312.14197*, 2025.
- [10] Z. Ying, H. Wang, J. Liu, Q. Zou, A. Liu, J. Yang, Y. Yang, and X. Liu. Agentvisor: Defending llm agents against prompt injection via semantic virtualization. *arXiv preprint arXiv:2604.24118*, 2026.

- [11] Z. Ying, X. Wu, H. Wu, X. Zheng, H. Cheng, X. Shi, and J. Guo. Skilljack: Persistent skill backdoors in self-evolving agents. *arXiv preprint arXiv:2608.03509*, 2026.
- [12] Q. Zhan, Z. Liang, Z. Ying, and D. Kang. Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 10471–10506, 2024.

A. Reproducibility and Artifact Map

The report was derived from the local experiment artifacts accompanying this technical report. The principal components are listed below.

Table 7 | Key local artifacts used for the DSH case study.

Artifact	Role
A.I.G assessment components	Payload construction, taint routing, and trace normalization
$\mathcal{J}_R / \mathcal{J}_L$	Deterministic and semantic outcome evaluation
full_channel_mode dataset	1,120-case text/file channel matrix
experiment runner	Matrix orchestration and online $\mathcal{J}_R$ evaluation
DSH runtime adapter	Python-to-TypeScript harness bridge and trace mapping
DSH driver	Real DSH runtime initialization and session-event export
controlled test plugin	Source fixtures and tracked simulated sinks
final trace corpus	One normalized trace for each planned agent run
offline judge corpus	Trace-level $\mathcal{J}_L$ verdicts

B. Interpretation Notes

All sink tools in this evaluation are simulated. A recorded sink call represents an attempted action by the agent; it does not send an email, execute a command, transfer funds, or otherwise affect an external system. Percentages are descriptive measurements for this controlled configuration, not claims of universal vulnerability rates for DeepSeek Harness or any model provider.